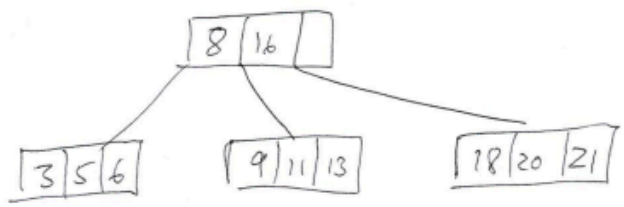
NYU School of Engineering May 10, 2026
Computer Science and Engineering
CS 6083, Spring 2026

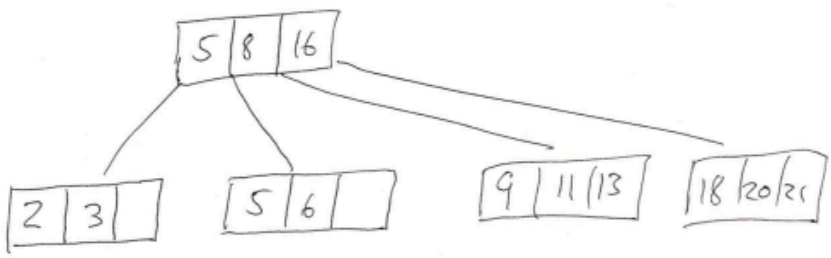
Problem Set #4 solution set

Problems 1:

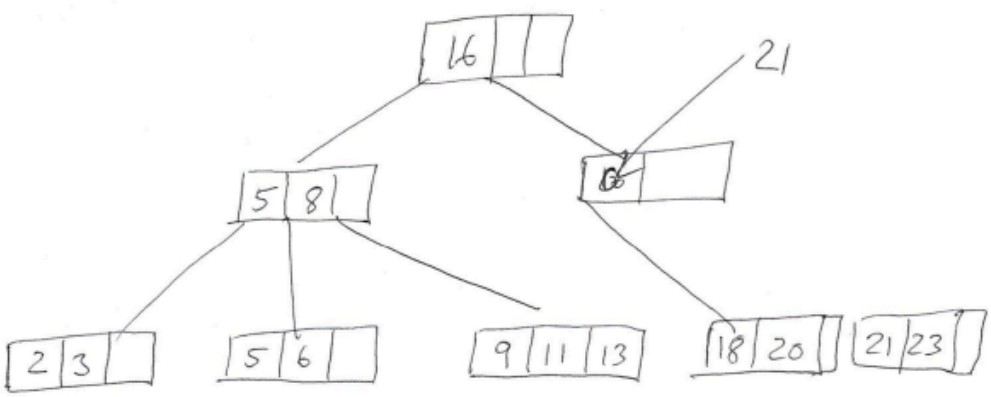
1 Insert 9 2 Insert 21



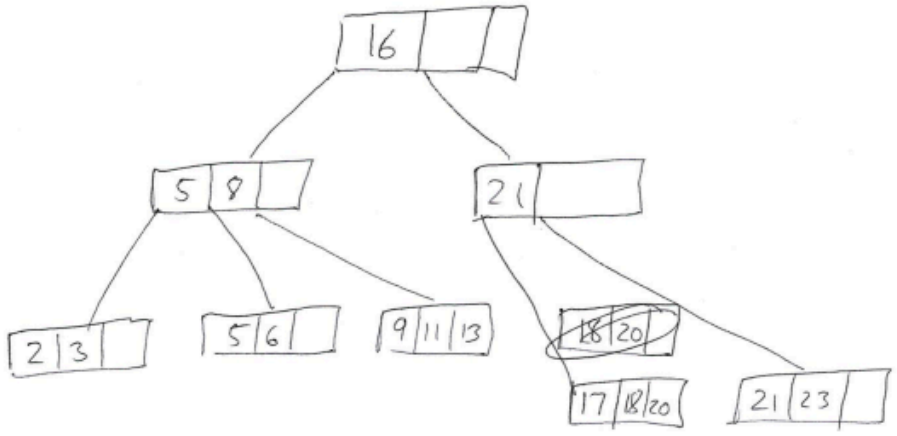
3 Insert 2



4 Insert 23

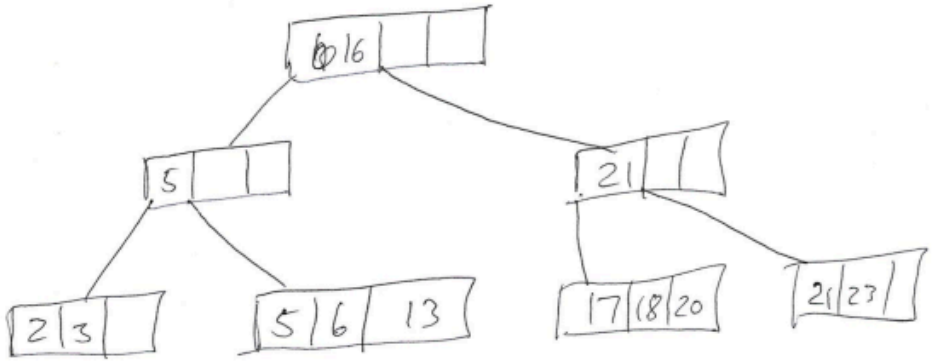


(5) Insert 17



(6) Delete 11 → just remove 11 from above

(7) ~~Remov~~ Delete 9

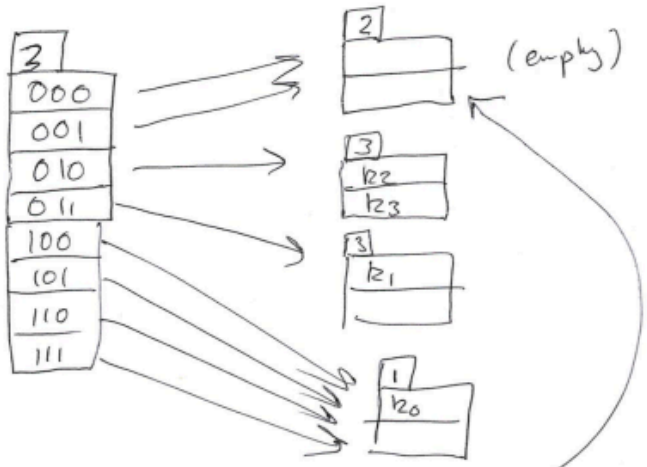


(8) Delete 18 → just remove 18 from above

Problem 2:

(2) Insert k_2 → just add k_2 to node with k_1

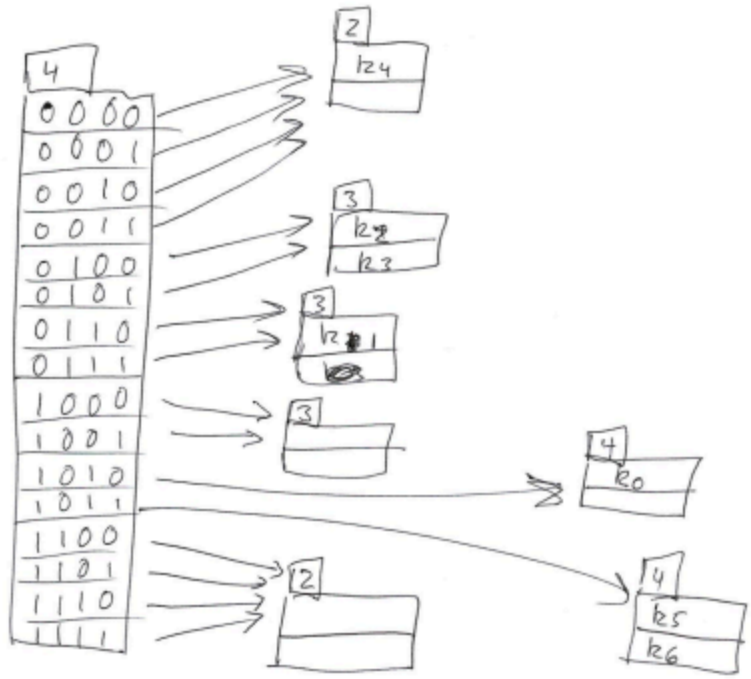
(3) Insert k_3



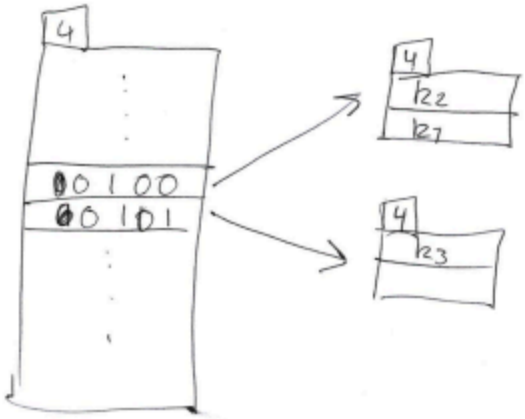
(4) Insert k_4 → just add k_4 here

(5) Insert k_5 → just add k_5 to node holding k_0

(6) Insert k_6



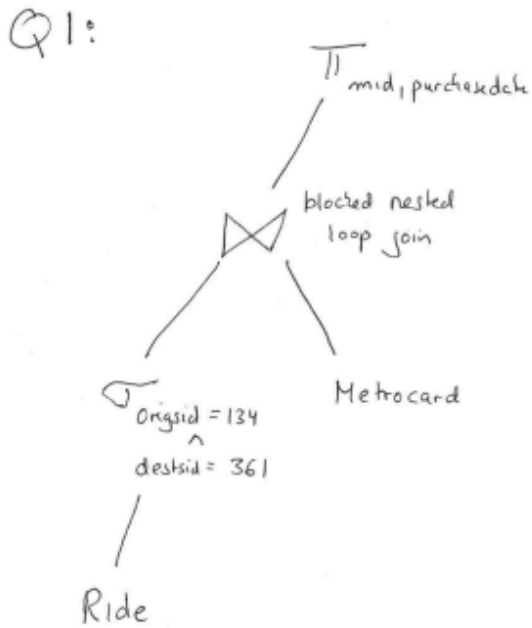
(7) Insert k_7 : change as follows in picture above

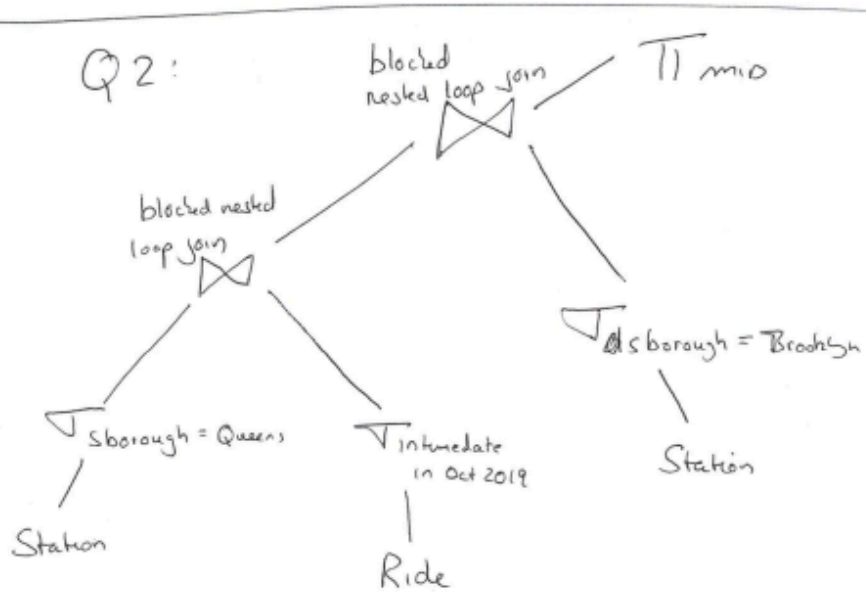


Problem 3:

(a) The first query gets the mid and purchase date of any metro card used to go from the station with ID 134 to the station with ID 351 (note that "destsid" is misspelled as "distsid"). The second query gets the mid of any metro card used to go from a station in Queens to a station in Brooklyn during October 2019.

(b) The query plans are as follows:





For the first query, we would scan the Ride table, and keep only those tuples with matching origin and destination stations. The Ride table has 5 attributes, thus 40 bytes per tuple. There are 10 billion tuples, resulting in 400GB of data, so it takes about 4000s to scan this table. After the selection for matching stations, we are left with 400GB * (1/500) * (1/500) = 1.6MB, since each station is responsible for about a 1/500 fraction of the total origin and destination traffic. Thus, this data fits can be kept in main memory, and we can do a blocked nested loop join that scans the Metrocard table only once, at a cost of 100M* 24B / (100MB/s) = 24s, for a total of 4024s.

For the second query, we scan Station, and keep only those entries where the station is in Queens and place them in main memory. We then start scanning the Ride table, at a cost of 4000s as above, to perform the blocked nested loop join. The output of this join is piped directly into the next join above it. Meanwhile, we also scan Station for entries in Brooklyn, and place those in main memory for the second blocked nested loop join. Thus, we can perform the second join directly as the output arrives from the first join, at no extra I/O cost. In fact, we could get away with scanning Station only once and checking for both Brooklyn and Queens. So total is 4000s plus cost of scanning Station, which is a tiny table of 16KB, so that does not add anything significant.

(c) For each node in the B+ tree there are n pointers and (n - 1) keys, thus we have $8 \times n + 8 \times (n - 1) = 4096$, or $n = 256$. We will round that to 250. With 80% occupancy in the index, this gives us n=200.

Dense Unclustered Index on mid in MetroCard

As there are 100 million tuples in the MetroCard table that each have an index entry. These 100M index entries require 500,000 index leaf nodes. At the next level, there are 2500 nodes, then 13, and then the root. Thus, the tree has 4 levels of nodes, it takes about 50ms to fetch a record using this tree, and the size of the tree is dominated by the leaf level, which is 500,000*4KB, or about 2GB.

Sparse Clustered Index origsid in Ride

Each tuple in the Ride table takes 40 bytes, and thus there are 100 tuples per page. Thus, there are 10B/100 = 100M index entries. Thus, the leaf level has 500,000 nodes as in the other case above, and the size and height of the index is also the same. Fetching one record will take 50ms, as before, but fetching 100,000 records will take 50ms plus the time to fetch 100,000 records of 40 bytes = 4MB, which is 40ms.

(d) For the first query, the most important thing would be to avoid the scan of the Ride table, so we would want index structures for origsid and destsid in Ride. This could be one composite index on (origdis, destsid), leaving us one more index structure to potentially speed up the join by avoiding a scan of the MetroCard table. However, note that after the section, we still have 40,000 Ride entries, and performing 40,000 index lookups would probably take longer than scanning the MetroCard table. Thus, one index is enough. The time for scanning Ride is reduced from 4000s to about 50ms+16ms.

For the second query, we could again want to avoid the scan of Ride during the join. This could be done by building an index on origsid or destsid in Ride. For example, we could use an index on origsid, so that after fetching the sids of all 100 stations in Queens, we would do 100 index lookups to fetch the data for rides originating in these stations, reducing cost by a factor of 5 over a full scan (given that there are 5 boroughs that each are assumed to have the same amount of traffic), from 4000s to 800s. An alternative would be to build on index on (origsid, destsid), to then determine all 10000 combinations of origsid in Queens and destsid in Brooklyn, and do 10000 lookups into Ride to fetch only those Rides, reducing cost of a scan to 4000s/25 = 16s plus the cost of 10000 index lookups, which is about 10000*50ms = 500s. The actual cost of the index lookups would probably be better than 500s given that many nodes in the index will be caching in memory.